

STL 排序

头文件: `#include<algorithm>`

默认排序: `sort(a+1, a+n+1);`

排序方法: 把a[1]到a[n]从小到大排序。

如果是排序a[0]--a[n-1]是

`sort(a, a+n);`

自定义排序:

```
bool compare(int a, int b) //默认a<b
{
    return a>b; //从大到小排序
}
sort(a+1, a+n+1, compare);
```

结构体排序

对于每一位同学，都有一个学号(num)和成绩(grade)。当成绩相同的时候按照学号的先后排序，当成绩不同的时候按照成绩从高到低排序。

```
struct st
{
    int num;
    int score;
};
```

//定义一个名字叫st的结构体

//st表示自定义数据类型，和int, char类似

//char, int是系统自带的数据类型

//而st是我们自己定义的数据类型

```
st s[100010]; //定义变量
```

//该变量里面有两个数据，num和score。

结构体排序

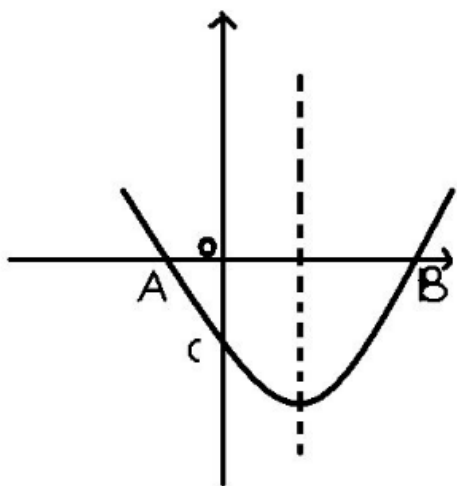
```
bool compare(st a,st b)
{
    if(a.score==b.score)
        return a.num<b.num;
    return a.score>b.score;
}

int main()
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
        scanf("%d%d",&s[i].num,&s[i].score);
    sort(s+1,s+n+1,compare);
}
```

sort默认排序方式是从小到排序，所以当定义好a和b之后，默认a<b(所有)，但是分数是从大到小排序，所以我们反向。学号从小到大排序，所以不变

二分，虽然效率非常高，但是他有一个特别大的限制，他要求序列或者答案是有序的才可以使用。如果当序列不是严格递增或者递减就没有办法使用。

以一元二次方程求解为例，一元二次方程是一个抛物线，如果要求该方程的零点，我们不能很简单的用 $f(n)*f(m)>0$ 表示没有零点， $f(n)*f(m)<0$ 表示有一个零点。这样思考是错误的。



方程的极值

给出一个 N 次函数，保证在范围 $[l, r]$ 内存在一点 x ，使得 $[l, x]$ 上单调增， $[x, r]$ 上单调减。试求出 x 的值。

输入格式：

第一行一次包含一个正整数 N 和两个实数 l 、 r ，含义如题目描述所示。

第二行包含 $N+1$ 个实数，从高到低依次表示该 N 次函数各项的系数。

输出格式：

输出为一行，包含一个实数，即为 x 的值。四舍五入保留5位小数。

输入：

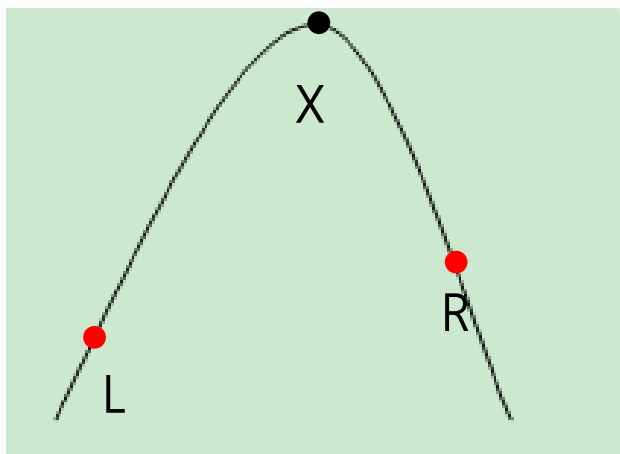
3 -0.9981 0.5

1 -3 -3 1

输出：

-0.41421

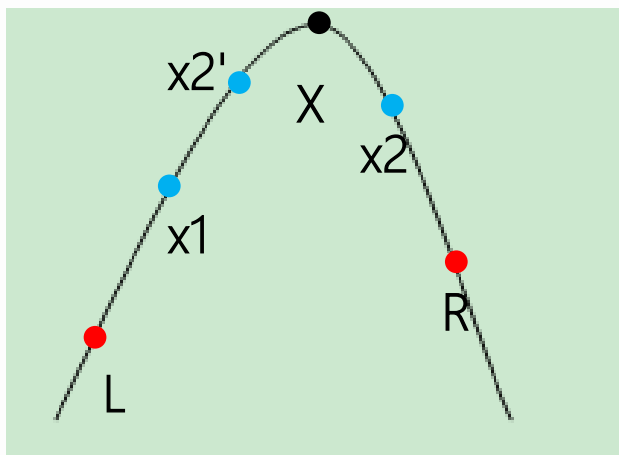
方程的极值



首先，这道题的答案（ x 的值）在一个已知的区间内，并且区间肯定是有顺序的，由于是高次方程，所以我们不知道该怎么用公式求解，所以只能一个一个去试答案。

因为要精确到小数点后五位，所以每次增加 $1e-7$ ，效率是比较低的，如果 l 和 r 的值稍微大一点，很容易超时，此时我们就想到了二分答案，找到 l 和 r 的中间点，然后。。。就没有然后了，判断中间点和两端的大小？明显不对。所以二分是不成立的。

方程的极值



如果只找一个点，没有与之做比较的，已知该图像在 $[l, r]$ 区间上是严格按照先增后减的规律的，所以我们可以找两个点 (x_1, x_2) ，并且 $x_1 < x_2$ ，然后判断这两个点的函数值的大小，然后缩小极的区间范围。

如果 $f(x_1) < f(x_2)$ ，说明 x_2 更加靠近极值点，因为 $x_1 < x_2$ ，所以 x_1 肯定在极值的左边，但是 x_2 不一定在极值的右边，所以可以缩小范围为 (x_1, r) 。

如果 $f(x_1) > f(x_2)$ ，说明 x_1 更加靠近极值点，因为 $x_1 < x_2$ ，所以 x_2 肯定在极值的右边，但是 x_1 不一定在极值的左边，所以可以缩小范围为 (l, x_2) 。

相等怎么办？

方程的极值

接下来是确定该怎么选择这两个点，即满足 $x_1 < x_2$ 的要求，又要让效率更高。即让区间的平均缩小速度最快。很明显就是区间的两个三等分点，这样选择的两个点每次把区间分为以前的 $2/3$ ，多次分割下来，就可以很快找到极值了。这种方法又叫做三分答案。

$$1.5^{35} > 1000000$$

```
int lmid=left+(right-left)/3;//左三等分点
int rmid=right-(right-left)/3;//右三等分点
if(f(lmid)<=f(rmid))//相等情况也是一样的
l=lmid;
else
r=rmid;
```